AI ENGINEERING

Stop Overpaying for Giant LLMs

1 Why Small Language Models (SLMs) win in production.

2 How to cut latency and cloud bills without losing accuracy.

SV
Srijan Verma
AI Systems Engineer

SWIPE →

The Overkill Bottleneck

1 Routing simple domain tasks to 70B+ models wastes budget.

2 High inference latency breaks real-time user experiences.

SV

Srijan Verma

AI Systems Engineer

The Practical Fix: SLM + RAG

1. Host a compact 3B-7B model using engines like vLLM.
2. Fetch dynamic context via vector search.
3. Use the SLM purely as a fast, context-aware reader.

SV

Srijan Verma

AI Systems Engineer

The Core Logic Pattern

1 Keep runtime logic lean and focused entirely on retrieved context.

main.py

```
def generate_answer(query):
    context = retrieve_context(query)
    prompt = f"Context: {context}\nQuery: {query}"
    inputs = tokenizer(prompt, return_tensors="pt")
    outputs = model.generate(**inputs)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```

SV

Srijan Verma

AI Systems Engineer

Real-World Engineering Gains

1 Drastically lower infrastructure and inference costs.

2 Full privacy and deployment freedom on private VPCs or edge hardware.

SV

Srijan Verma

AI Systems Engineer

The Takeaway

1. Default to domain SLMs for 90% of repeatable tasks.

2. Follow Srijan Verma for pragmatic backend and AI architecture.

SV

Srijan Verma

AI Systems Engineer

FOLLOW +